IFCD107 · ESPECIALISTA EN INTELIGENCIA ARTIFICIAL

Un árbol de preguntas de sí o no

Sobreajuste, poda e importancia de variables

Sprint 8 · S08_01 · 35 min de cuaderno en clase (ejercicios 1, 2 y 3)

LA IDEA · PARTES DE UN ÁRBOL

Preguntas de sí o no encadenadas

Raíz · ¿antelación ≤ 115 días?

Las 10 reservas: 5 cancelan y 5 no.

Sí → rama · ¿va sin depósito?

6 reservas: 1 cancela.

No → hoja: cancela

4 reservas: las 4 cancelan.

No → hoja: no cancela

3 reservas con depósito: ninguna cancela.

Sí → hoja: no cancela

3 reservas sin depósito: 1 cancela.

Cada **hoja** predice la clase **mayoritaria** de las reservas que caen en ella.

LA IDEA · CÓMO ELIGE LA PREGUNTA

La mejor pregunta es la que más purifica

Gini mide cuánto se mezclan las clases en un grupo: **0** = todas iguales (hoja pura), **0,5** = mitad y mitad. Con 6 reservas y 1 cancelada: $1 - (1/6)^2 - (5/6)^2 = 1 - 0{,}03 - 0{,}69 = \mathbf{0{,}28}$.

Pregunta	Grupo «sí»	Grupo «no»	Gini ponderado
Sin preguntar	10 reservas · 5 cancelan	—	0,50
¿Antelación ≤ 115 días?	6 reservas · 1 cancela · 0,28	4 reservas · 4 cancelan · 0,00	**0,17**
¿Va sin depósito?	6 reservas · 4 cancelan · 0,44	4 reservas · 1 cancela · 0,38	0,42

Gana la pregunta que deja el menor Gini ponderado: la de la antelación, 0,17 frente a 0,42.

Ponderado = cada grupo cuenta según su tamaño: $(6 \times 0{,}28 + 4 \times 0{,}00) / 10 = 0{,}17$.

EJERCICIO 1 · LEER EL ÁRBOL

Cada línea es una pregunta; cada class, una hoja

```
print(export_text(arbol, feature_names=list(nombres)))
|--- antelacion <= 115.00
|   |--- sin_deposito <= 0.50
|   |   |--- class: 0
|   |--- sin_deposito >  0.50
|   |   |--- class: 0
|--- antelacion >  115.00
|   |--- class: 1
```

Sangría = profundidad

Lo que cuelga de una pregunta es su respuesta: **<=** sí, **>** no.

Columnas 0 / 1

Tras el one-hot, **<= 0.50** = **no** y **> 0.50** = **sí**.

class: 0 = la hoja predice «no cancela» · **class: 1** = «cancela».

```
nombres[arbol.tree_.feature[0]]   # 'antelacion' la variable de la raíz
arbol.get_n_leaves()              # 3 nº de hojas
```

EJERCICIO 2 · SOBREAJUSTE

El árbol libre memoriza

Sin límites, el árbol sigue preguntando hasta que **cada hoja es pura**. Es como aprenderse de memoria los exámenes de otros años: un 10 en esos… y un suspenso en el nuevo.

69

hojas

para 200 reservas de entrenamiento

100 %

de aciertos en train

las memoriza todas, ruido incluido

67 %

de aciertos en test

reservas que no había visto

```
arbol_libre = DecisionTreeClassifier(random_state=SEMILLA).fit(Xtr, y_train)
roc_auc_score(y_test, arbol_libre.predict_proba(Xte)[:, 1])
```

Señal de alarma: nota perfecta en train y mucho peor en test. En el cuaderno: AUC ≈ 1 en train, menos de 0,70 en test.

EJERCICIO 3 · PODA

Podar: frenar el crecimiento

max_depth

Profundidad máxima: **cuántas preguntas seguidas**.

min_samples_leaf

Mínimo de reservas por hoja: evita hojas «de 2 casos».

ccp_alpha

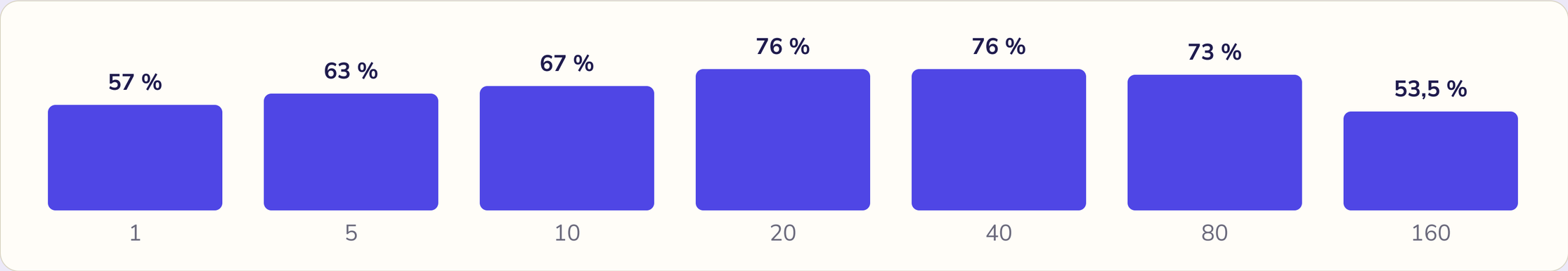
Poda **a posteriori**: se recortan las ramas que no compensan su complejidad.

Mismo conjunto de datos	Hojas	Acierto en train	Acierto en test
Árbol libre	69	100 %	67 %
min_samples_leaf = 5	26	81 %	70 %
min_samples_leaf = 20	7	76,5 %	74,5 %
max_depth = 2	4	77 %	74,5 %

Como podar un rosal: se quita lo que sobra para que la planta crezca mejor. Menos hojas, **menos distancia** entre train y test.

EJERCICIO 3 · ACIERTO EN CV SEGÚN MIN_SAMPLES_LEAF

Se elige con validación cruzada, nunca con test




```
hojas_min = [1, 5, 10, 20, 40, 80, 160]
acierto = [0.57, 0.63, 0.67, 0.76, 0.76, 0.73, 0.535]   # uno por cada valor
hojas_min[int(np.argmax(acierto))]                       # 20, el primer máximo
```

Demasiada poda (160) deja el árbol corto. El test se mira una sola vez, al final, con el árbol ya elegido.

PUNTOS FUERTES

Un árbol no necesita escalar

Mismo árbol, dos unidades	Antelación en días	Antelación en semanas (días ÷ 7)
Pregunta de la raíz	¿antelación ≤ 115?	¿antelación ≤ 16,4?
Reservas en cada grupo	6 y 4	6 y 4
Predicciones de las 10 reservas	las mismas	las mismas

Por qué

Un árbol solo **compara con un umbral**. Cambiar la unidad mueve el umbral, pero no el orden de las reservas.

Ojo

Con k-NN o la regresión logística, escalar **sí** importaba. Con árboles, solo hace falta imputar nulos y codificar categorías.

Fuerte

Explicable: se lee como reglas.

Débil

Inestable y suele rendir menos que los ensembles.

(CASA) · EJERCICIO 4 · IMPORTANCIA DE VARIABLES

Cuánto purifica cada variable en total

Pregunta del árbol	Impureza antes	Impureza después	Reducción
Raíz: antelación	10 × 0,50 = 5,00	6 × 0,28 + 4 × 0 = 1,67	3,33
Rama: sin depósito	6 × 0,28 = 1,67	3 × 0 + 3 × 0,44 = 1,33	0,33

Total = 3,67 → antelación **91 %** (3,33 / 3,67) y sin depósito **9 %** (0,33 / 3,67). La suma es 1.

```
imp = pd.Series(arbol.feature_importances_, index=nombres).sort_values(ascending=False)
imp.round(2).to_dict()           # {'antelacion': 0.91, 'sin_deposito': 0.09}
top3 = list(imp.head(3).index)   # los 3 nombres más importantes
```

No es causal

Dice qué **mira el modelo**, no qué causa la cancelación.

Infla numéricas

Favorece variables con **muchos valores** distintos.

Correlacionadas

Se reparte de forma algo **arbitraria**.

(CASA) · INESTABILIDAD

Pequeños cambios en los datos, otro árbol

5 árboles, cada uno con el 80 % de train	Árbol 1	Árbol 2	Árbol 3	Árbol 4	Árbol 5
Umbral de la raíz (días de antelación)	111,6	111,6	111,6	112,5	108,4
Hojas	12	13	12	13	14

Misma configuración, otros datos: en el 41 % de las 200 reservas de prueba, los 5 árboles no coinciden.

```
predicciones = np.array(predicciones)          # una fila por árbol
(predicciones.min(axis=0) != predicciones.max(axis=0)).mean()
```

Lo arreglan los **ensembles**: bosques aleatorios (S08_02) y boosting (S08_03).

(CASA) · MINI-RETO

Del árbol a frases para el gerente

Ruta de la raíz a la hoja	Reservas	Cancelan	Frase para el gerente
antelación > 115	4	4 (100 %)	«Con más de 115 días de antelación, canceló el 100 % (4 de 4).»
antelación ≤ 115 y con depósito	3	0 (0 %)	«Con poca antelación y depósito, nadie canceló (0 de 3).»
antelación ≤ 115 y sin depósito	3	1 (33 %)	«Con poca antelación y sin depósito, canceló 1 de cada 3.»

```
arbol_gerente = DecisionTreeClassifier(max_depth=3, min_samples_leaf=50, random_state=SEMILLA)
print(export_text(arbol_gerente, feature_names=list(nombres)))
```

Una regla es **un camino** de la raíz a una hoja. Añade siempre una cifra y escribe con palabras de negocio.

HOY HAS APRENDIDO

Preguntas, poda y reglas

Un árbol

Preguntas de sí o no encadenadas que buscan la mayor pureza en cada rama

Sobreajuste y poda

Sin límites **memoriza**; **max_depth**, **min_samples_leaf** y **ccp_alpha** lo frenan

Fuerte y débil

Explicable y sin escalado, pero **inestable** y menos preciso que los ensembles

Siguiente: S08_02 · Random Forest, la sabiduría de muchos árboles